Übungsblatt 8: Besitz, Fortschritt und Fehlerreaktion

Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Ziele und Ablauf

Sie erklären Konkurrenzfehler, entwerfen ein Warteprotokoll, berechnen Watchdog-Zeitfenster und beurteilen Speicher- sowie Ausgangsschutz. Grundlage sind Vorlesungen 1–7 und Vorlesung 8. HSEM bedeutet Hardware Semaphore, IWDG Independent Watchdog, WWDG Window Watchdog, EWI Early Wakeup Interrupt und MPU Memory Protection Unit.

Plan: vier Papieraufgaben mit insgesamt **32 Punkten in 50 Minuten**, anschließend **25 Minuten** im vorbereiteten STM32-Labor. Das Abschlussquiz benötigt weitere 15 Minuten. Alle Papieraufgaben sind ohne Board lösbar. Alle Zahlen sind Lehrannahmen. Messfelder bleiben leer, bis Sie selbst gemessen haben.

Aufgabe 1 · Konkurrenz und Endwert · 8 Punkte · 12 Minuten

Zwei Kerne sollen einen gemeinsamen Zähler von fünf aus jeweils einmal erhöhen. Einzelne ausgerichtete Wortzugriffe gelten im Modell als ungeteilt. Lesen, Addieren und Schreiben sind getrennte Schritte.

1. Geben Sie eine Reihenfolge mit Endwert sechs an. Nennen Sie die privaten Kopien und den gemeinsamen Speicher. Begründen Sie, warum `volatile` und ein Schloss nur um das Schreiben nicht ausreichen. **(3 P)**
2. Ein neuer endlicher Lauf startet bei null. Je Kern werden 100 Erwerbsversuche ausgeführt. M7 meldet 97 und M4 95 erfolgreiche Inkremente. Beide Abschlussmeldungen sind sichtbar, der Speichervertrag gilt und es gibt keinen Überlauf. Was ist der Sollwert? Beurteilen Sie auch einen ungeschützten Lauf, der zufällig diesen Wert erreicht. **(2 P)**
3. Zwei M4-Tasks benutzen dieselbe HSEM-Nummer und beide FastTake. Warum reicht das nicht? Was bedeutet ein zweites HAL_OK desselben Besitzers, und wie wirken eine einzelne Freigabe und ein zusätzliches lokales Mutex? **(3 P)**

Ihre Lösung

(Notieren Sie Ihre Schritte und Begründungen hier.)

Aufgabe 2 · Aufwecken, Frist und Sperrordnung · 8 Punkte · 12 Minuten

Ein M4-Task scheitert beim Take. M7 gibt frei, bevor M4 seine Notification aktiviert. Erst danach ruft M4 Wait auf. Modell: genau ein wartender Task, ein gespeichertes Thread-Flag und keine weitere garantierte Freigabe.

1. Warum kann der Task unnötig blockiert bleiben? **(1 P)**
2. Geben Sie die richtige Reihenfolge aus Scharfschalten, Take, Flagbehandlung, Wait und erneutem Take an. Die Referenz-HAL deaktiviert ausgelöste Notification-Bits im Handler. Berücksichtigen Sie das und nennen Sie die Rolle der zulässigen Interruptpriorität. **(3 P)**
3. Der gesamte Versuch beginnt bei 0 ms und hat 20 ms Frist. Meldungen kommen bei 8 und 17 ms. Ein erneuter Take scheitert bei 9 ms. Welche Restzeiten gelten? Warum wäre jeweils ein neues 20-ms-Warten falsch? **(2 P)**
4. M7 hält Schloss 1 und will 2, M4 hält 2 und will 1. Nennen Sie eine vorbeugende Entwurfsregel und erklären Sie, warum ein Timeout allein die bereits gehaltenen Schlösser nicht löst. **(2 P)**

Ihre Lösung

(Skizzieren Sie das Protokoll und die Restzeiten hier.)

Aufgabe 3 · Zwei Watchdog-Rechnungen · 8 Punkte · 14 Minuten

Für den WWDG2 gelten hier $f_{\mathrm{PCLK}} = 100\,\mathrm{MHz}$, Vorteiler $P = 8$, Nachladewert $R = 127$ und Fenster $W = 80$. Erlaubt ist $63 < T \leq 80$. EWI wird bei $T = 64$ angefordert, Reset beim Schritt nach 63. **Idealphase:** erster Abwärtsschritt einen vollen Tick nach dem Start.

1. Berechnen Sie mit $\Delta t = 4096P/f_{\mathrm{PCLK}}$ den Tick und die idealen Zeiten bis Fensteröffnung, EWI und Reset. Ist Gleichheit $T = W$ erlaubt? **(3 P)**
2. Bewerten Sie Refresh nach 5, 18 und 25 ms. Warum ist `osDelay(18)` ohne weitere Bedingungen kein Nachweis von 18 ms tatsächlicher Wartezeit? **(1 P)**
3. Beim IWDG ist das Fenster ausgeschaltet. $R = 500$, $P = 32$. Nehmen Sie rein didaktisch $f_{\mathrm{LSI,min}} = 28\,\mathrm{kHz}$ und $f_{\mathrm{LSI,max}} = 36\,\mathrm{kHz}$ an. Berechnen Sie früheste und späteste Ablaufzeit mit $(R+1)P/f$. Welche davon ist für rechtzeitigen Refresh maßgeblich? **(2 P)**
4. Die EWI-Reaktion hat nominal einen WWDG-Tick. Angenommene Eintrittslatenz 40 µs, Handler-Grundarbeit 20 µs, Reserve 30 µs. Welches Arbeitsbudget bleibt? Nennen Sie eine Bedingung, unter der der Callback gar nicht rechtzeitig läuft. **(2 P)**

Ihre Lösung

(Zeigen Sie Einheiten, Rechenweg und Annahmen hier.)

Aufgabe 4 · Fortschritt und Ausgangsschutz · 8 Punkte · 12 Minuten

Ein Supervisor hat bei jetzt 25 ms zuletzt neue Epochen von Worker A bei 20 ms und von B bei 10 ms gesehen. A hat eine Frischefrist von 10 ms, B von 30 ms. Alle Zeiten benutzen die lokale Supervisor-Uhr.

1. Sind beide Nachweise frisch? Was beweist eine weiterlaufende Idle-Task bei zwei blockierten Workern? **(2 P)**

2. Für die Reaktion ab tatsächlichem Stillstand gelten die angenommenen Höchstwerte: Frischefrist 30 ms, Abfrageverzug 5 ms, möglicher Meldeverzug 2 ms, Watchdogrest 21 ms, Startreaktion 8 ms. Reicht dies für eine Anforderung von 60 ms? **(2 P)**
3. Eine passende No-Access-Region ist auf M7 aktiv, ohne erlaubende Überlappung. Wird ein privilegierter gewöhnlicher Interrupt-Handler daran gehindert zu schreiben? Gilt dieselbe Region für DMA oder M4? Ein Arrayüberlauf landet innerhalb einer erlaubten Region: muss er faulten? **(2 P)**
4. Ein isoliertes aktives High-Signal ENABLE wurde im EWI auf Low gesetzt. Erklären Sie, warum zusätzlich der Pegel während Reset und Boot geregelt werden muss. Nennen Sie eine passende Lehrbeschaltung und eine Bootregel. **(2 P)**

Ihre Lösung

(Begründen Sie jede Schutzgrenze hier.)

Labor A · Der endliche Zähler · 25 Minuten

Verwenden Sie **STM32CubeMX und VS Code mit STM32 VS Code Extension für ein NUCLEO-H755ZI-Q**. M7 bleibt ohne Kernel, M4 verwendet FreeRTOS über CMSIS-RTOS2. Ein bereits gemeinsam startendes Projekt und die Speicherbasis aus Vorlesung 6 müssen vorbereitet sein. Die vollständige Integration steht in code/lab08/README.md.

1. Übernehmen Sie `lab08_counter.h`, `lab08_counter.c`, `lab08_shared.c` in beide tatsächlichen CMake-Ziele. Definieren Sie pro Ziel genau `CORE_CM7` oder `CORE_CM4`. Entfernen Sie vorherige Labor-Anwendungsobjekte. Reservieren Sie HSEM 1 ausschließlich für je einen Worker pro Kern.
2. Der neue Shared-Block **ersetzt** den Vorlesung-6-Block: 28 Byte Nutzstruktur, 32 Byte Section, SRAM4 ab `0x38000000`. Übernehmen Sie `shared-section.ld` in beide tatsächlich verwendeten Linkerskripte, ohne Speicheralias. Prüfen Sie die beiden ELF-/Map-Dateien und dass Startcode/Debugger die Section nicht nachträglich nullen.
3. M7 konfiguriert bei ausgeschaltetem Datencache eine tatsächlich freie MPU-Region als Normal, nicht cachebar und führt `Lab08_CM7_InitShared(LAB08_PROTECTED)` aus, während M4 noch im vorhandenen Boot-Stop wartet. Anschließend darf der Cache an, dann folgt die **bestehende** Bootfreigabe. M7 ruft danach einmal `Lab08_CM7_Run()` auf. Der einzige M4-Worker ruft `Lab08_CM4_Run(argument)` auf.
4. Bauen und programmieren Sie **beide Images**, starten Sie gemeinsam und lassen Sie den endlichen Versuch frei laufen. Erst bei M7-`lab08_complete=1` halten Sie beide Kerne an. Notieren Sie Treffer, Fehlschläge, Fehlercode, Endwert und Erfolgssumme. Bei Timeout kein Ergebnis als abgeschlossenen Lauf bewerten.
5. Für den Vergleich ändern Sie nur die einmalige M7-Initialisierung auf `LAB08_UNPROTECTED`. Beide Images neu bauen/programmieren, gemeinsam neu starten, dieselben Werte notieren. Der Shared-Modus wird von beiden gelesen. Interpretieren Sie auch den möglichen zufällig passenden Endwert. Nie den Zähler bei laufendem Partner neu initialisieren.

Projektbedingung	Ihre Angabe
Boardrevision, CubeMX / HAL / Extension-Version	
beide Kern- und Bustakte, M4-Kerneltick	
Boot-HSEM / Zähler-HSEM / freie MPU-Region	
Shared-Adressen, Größe, Starttabellen geprüft	

Lauf	Treffer M7 / M4	Fehlschläge	Endwert / Sollsumme	Fehler / Erklärung
geschützt				
ungeschützt				

Vorbereitete Zusatzstation · WWDG2 und isolierter Ausgang

Diese Station ergänzt den Zählerversuch, wenn ein zweites fertig vorbereitetes Projekt und ein Logikanalysator verfügbar sind. Sie benötigt keine neue Pflicht-Toolchain. Bei engem Zeitbudget zeigt die Lehrperson die drei Fälle. WWDG2 läuft auf M4 in einer **separaten Projektkopie**, ohne Zählerworker und ohne fremdes Füttern. Kein Aktor wird angeschlossen.

- Nutzen Sie `lab08_wwdg.h`, `lab08_wwdg.c`, `lab08_board.c` nur im M4-Ziel. Das Modul startet WWDG2 erst im laufenden Task. Entfernen Sie einen vorherigen generierten `MX_WWDG2_Init()`-Aufruf. APB1-PCLK 100 MHz, Kerneltick 1 kHz, Teiler 8, Fenster 80, Reload 127, EWI an. Andere Watchdogs und deren Startoptionen sind aus.
- Wählen Sie anhand Boardplan drei freie GPIOs mit CubeMX-Labels `LAB_ENABLE`, `LAB_REFRESH`, `LAB_EWI`. Ausgangsstart Low, Takte an, nur isolierte Anzeige/Messpunkte. ENABLE ist aktiv High und benötigt einen externen Pull-down für die Resetphase. Die Anleitung enthält vollständige GPIO-Funktionen und verlangt bewusst keine angenommene Boardpinzahl.
- Die generierte `WWDG_IRQHandler` ruft genau einmal `Lab08_WWDG_IRQHandler()` auf. EWI gehört laut H755-Header zu `WWDG_IRQn`, nicht zu `WWDG_RST_IRQn`. Die Projektpriorität wird in einer statischen `Lab08_WWDG_Config` angegeben. Der Callback benutzt keine Kernel-API, kein Delay und keinen Refresh.
- Testen Sie je Neustart genau einen Modus: `LAB08_WWDG_EARLY` (5 Ticks), `LAB08_WWDG_VALID` (zehn absolute 18-Tick-Ziele, dann bewusst nicht weiterfüttern), `LAB08_WWDG_LATE` (25 Ticks ohne Refresh). Das Modul prüft im gültigen Fall zusätzlich den Zähler vor dem Schreiben. Die kurze Interruptmaskierung hält nicht den Hardwarezähler an und ist kein Timingbeweis.
- Beobachten Sie Marker direkt am Vorgang. EWI kann bei frühem Reset fehlen. Ergänzen Sie `lab08_reset.h/.c` nur im M7-Ziel und rufen Sie `Lab08_CM7_CaptureReset()` früh vor HAL-/Cacheinitialisierung auf. M7 sichert und löscht als einziger die gemeinsame Resetursache; M4 löscht sie nicht. Dokumentieren Sie Debug-Freeze, tatsächlichen freien Lauf und Pinpegel während Reset/Boot. Keine langen Haltepunkte im laufenden Fenster. Diagnosewörter sind nach Reset kein automatisch erhaltenes Log.

Fall	Modell	Resetursache / Markerzeit / Pegel beobac
früh	Refresh außerhalb Fenster, EWI nicht erforderlich	
gültig	zehn akzeptierte Refreshversuche, danach geplanter Reset	
spät	EWI falls bedient, anschließend Reset vor 25 Ticks	

Abgabe und Aussagegrenzen

Geben Sie die vier Papierlösungen, Ihre beiden Build-/Layoutbefunde und die tatsächlich ausgefüllten Beobachtungstabellen ab. Kennzeichnen Sie Modellrechnung, Quellprüfung und Boardmessung getrennt. Falls Sie keine Hardware ausführen konnten, dokumentieren Sie dies und geben Sie keine ausgedachten Messwerte an. Ein optionaler MPU-Test benötigt eine

eigens vorbereitete No-Access-Adresse, aktiven Schutz, sichtbaren Maschinenzugriff und einen tragfähigen Fault-Stack; er wird nicht mit dem Watchdogfenster kombiniert.